

Benchmarking Service

Architecture & Design Overview

A pure-Python, HTTP-only service that deploys and evaluates agent benchmarks across multiple cluster-specific Rossoctl / Kagenti instances.

Overview & Key Design Decisions

What it is

Deploys a benchmark's MCP tool + A2A agent, runs the evaluation, reads results, and exports them — all over HTTP.

Objectives

- Automate benchmarking lifecycle operations
- Cross-user/cluster sharing of benchmark run results
- Easy to configure in-cluster & cross-cluster benchmark runs
- Asynchronous parallel benchmark runs
- Secure, scalable, auditable & resilient

3 benchmarks

- gsm8k (single-turn) · tau2 (multi-turn, server-side simulator) · appworld

Client contract

- Point at a hostname, send Authorization: Bearer <caller JWT> — same from kind to prod

Key design decisions

Pure-Python, HTTPS-only to Rossoctl

No shell-out: smaller image, no injection surface, structured errors, testable.

Two-token auth model

Caller JWT attributes + routes only (never forwarded); Service mints its own benchmarker (ROPC) token to Rossoctl.

iss is the trust anchor

The JWT issuer selects the per-instance config — no separate instance argument, so no confused-deputy escape.

Per-request ROPC login

Fresh Service token every request — no expiry handling.

iss-keyed per-instance config

One file per issuer: Rossoctl URL, benchmarker cred, MLflow/S3, optional Keycloak backchannel + workload route templates.

Enact only what HTTP allows

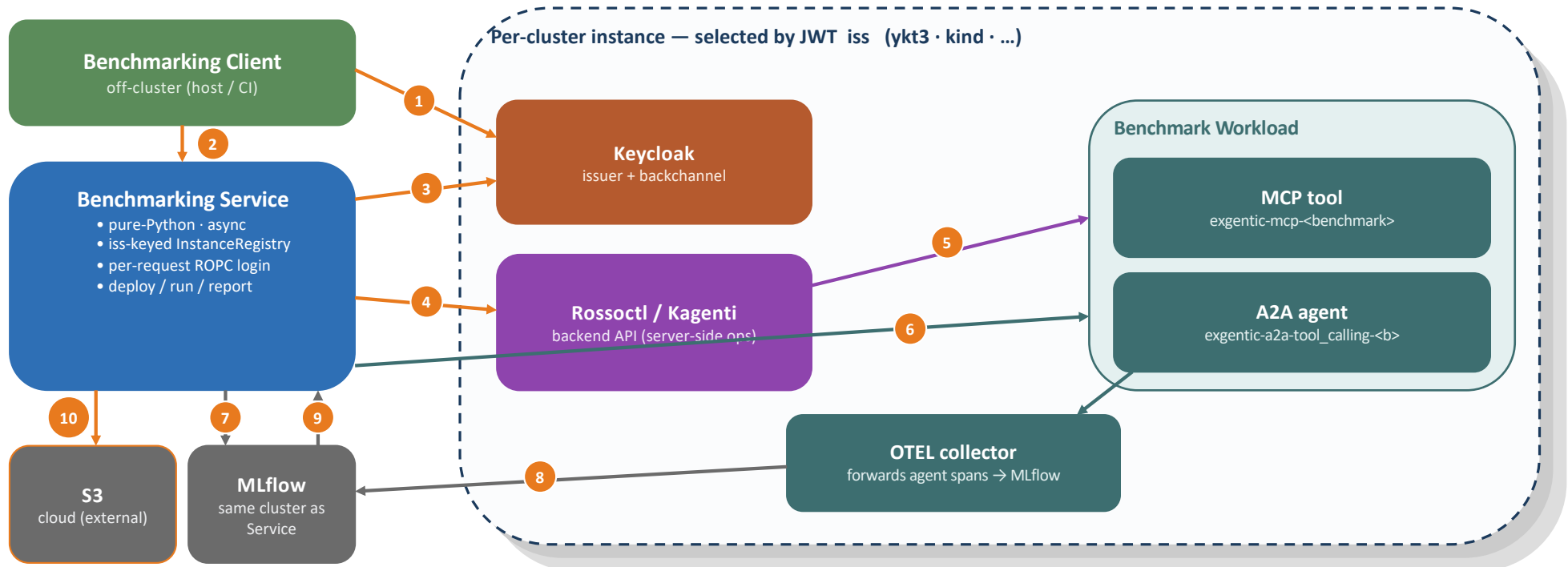
Deploy/run/report/export; workload Secrets + AuthBridge layer-3 are out-of-band → prechecked/rejected, never silently ignored.

MLflow with Service; S3 in the cloud

MLflow is co-located with the Service (per-service traces it emits + reads), not in the workload cluster; S3 is an external cloud service — the shared cross-service sink.

Architecture

Relations between client, service, cluster-specific Keycloak / Rossoctl / workload, MLflow & S3



- 1 Client logs in to Keycloak (ROPC) → caller JWT
- 2 Client → Service: Authorization: Bearer <caller JWT>
- 3 Service validates JWT (JWKS) + mints its own benchmarker token (ROPC, via backchannel)
- 4 Service → Rossoctl: deploy / list agents + tools
- 5 Rossoctl creates the MCP tool + A2A agent in-cluster

- 6 Service runs benchmark sessions directly (MCP / A2A)
- 7 Service emits the Agent.Session trace → MLflow (first)
- 8 A2A agent → OTEL collector → MLflow (optional; off by default)
- 9 Service reads back Agent.Session traces (MLflow)
- 10 Service exports run.json / report.* (S3)

The Two-Token Auth Model

Why the caller's token is never forwarded upstream

Caller JWT (inbound)

Presented by the client as Authorization: Bearer.

- Used ONLY to attribute (preferred_username) and route (iss → instance)
- Signature validated via the issuer's JWKS
- aud / exp deliberately lenient in the dev/ops context
- **NEVER forwarded to Rossoctl**

benchmarker is special:

- A caller JWT with preferred_username == benchmarker authorizes config ops (GET/PUT /config)

Service token (outbound)

Minted by the Service per request via ROPC (password grant) using the instance's benchmarker credential.

- Always fresh → no expiry handling
- Presented to Rossoctl for all cluster-facing ops
- **The Service acts under its OWN identity, not the caller's**

Backchannel split (iss ≠ dialed host):

- When the iss host is unreachable (in-cluster kind), JWKS + token come from a configured backchannel URL; iss is matched, never dialed

Implication: at the Rossoctl / cluster layer every action appears as the Service's identity — so per-user attribution & audit live in the Service, keyed on (iss, preferred_username).

Benchmark Catalog & Run Lifecycle

Catalog (static registry)

gsm8k

single-turn · needs hf-secret + openai-secret

tau2

multi-turn · user-simulator LLM runs server-side in the MCP pod · raise timeout

appworld

declared + deploys; MCP image blocked by its own venv-service timeout

Lifecycle (REST)

POST /benchmarks/{n}/deploy

create MCP tool + A2A agent → 201

GET /benchmarks/{n}/status

poll until tool_ready & agent_ready

POST /benchmarks/{n}/runs

precheck → 202 run_id (409 not deployed · 424 secret missing)

GET .../runs/{id}

poll pending → running → succeeded / failed

GET .../runs/{id}/report

MLflow records + artifacts (409 if MLflow unset)

download from S3

run.json · report.ndjson · report.parquet

What the Service Can & Cannot Enact

The HTTP-only boundary, made explicit

✓ Enacts over HTTP

- Deploy MCP tool + A2A agent (with CPU/mem, image, env)
- Deploy-time model swap (per-experiment agent)
- authbridge_enabled → inject sidecar w/ cluster-default pipeline (layer-2)
- Run benchmark sessions; collect pass/fail + latency
- Emit + read MLflow traces; export to S3
- Service-owned config: MLflow + S3 via PUT /config (benchmarker only)

✗ Out-of-band (reports, doesn't do)

- Cluster Secrets (hf-secret, openai-secret) — operator provisions; run precheck returns 424 naming the missing secret
- AuthBridge layer-3 plugin composition / on_error — needs a per-agent ConfigMap kubectrl overlay; deploy returns 422
- Any cluster-level API call — Rossoctl does these server-side

Principle:

- Never silently ignore an un-enactable request — precheck (424) or reject (422) with an actionable reason.

Cluster-agnostic by construction: works on kind / vanilla k8s / OpenShift; cross-cluster runs use per-instance route templates + a reachable internal issuer.